A1: Neural Texture Compression

Author: Vimal Mollyn
andrew: vmollyn

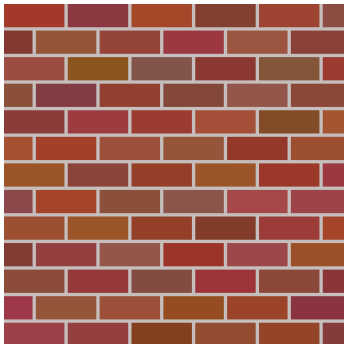
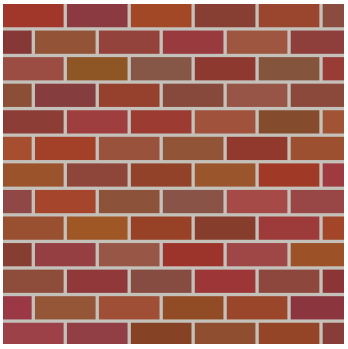
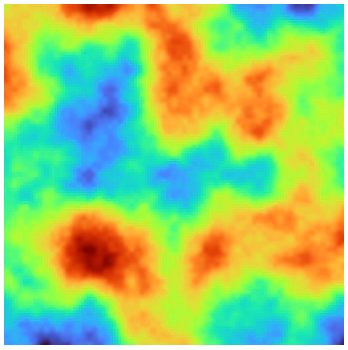
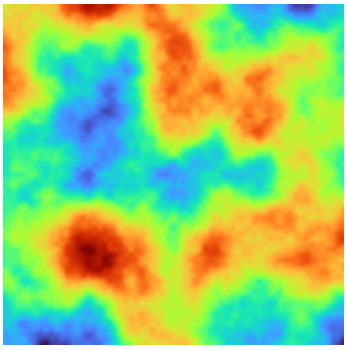
Github: https://github.com/VimalMollyn/15-674-Neural-Graphics

---

[P1] Texture Sampler (Bilinear): In code

[P2] S3TC (DXT1) Baseline

Reconstruction next to original for all 3 textures, with PSNR.

PSNR (dB)	S3TC	Original
41.13		
35.87		
42.58		

Compression Ration and PSNR

PSNR noted above.

The compression ratio is the same for all textures, since they all have the same original size of 512x512.

Each 4x4 block of the texture originally uses 4x4x3x1 = 48 bytes

After S3TC compression, each block use 4 bytes for the palette (2 bytes per color x 2 colors C0, C1) + 2 bits per texel x 4x4 = 8 bytes per block

So the compression ratio is 8/48 = 0.167.

How did I choose the endpoint colors?

I choose the endpoint colors as the minimum and maximum R,G,B values of all the colors in the block. This is easy to compute, and assumes that the color palette lies on a straight line that spans a diagonal line on the RGB cube. However, due to this assumption, colors that fall too far off this diagonal will not be represented easily.

Where does S3TC visibly break down?

a) From the PSNR values, it seems that this method works well for gradient.png and bricks.png. To my eye, I think it works best for bricks.png, then clouds.png and finally gradient.png, which clearly shows the limits of the PSNR metric.
b) S3TC seems to visibly break down to my eyes in the the gradient.png and clouds.png. This is because of two reasons. First, for the gradient.png the effects of quantizing the colors is clearly visible, we can see blocks in the compressed image. Second, the effects of choosing a limited color palette are visible in the clouds.png image, where there is high frequency color differences in each block. In contrast, the 4x4 blocks in bricks.png usually have 1 or maybe 2 colors solid colors in each block, and hence the compression is highly effective.

The endpoints are stored in RGB565, which gives green one more bit than red or blue. Explain why the extra bit goes to green.

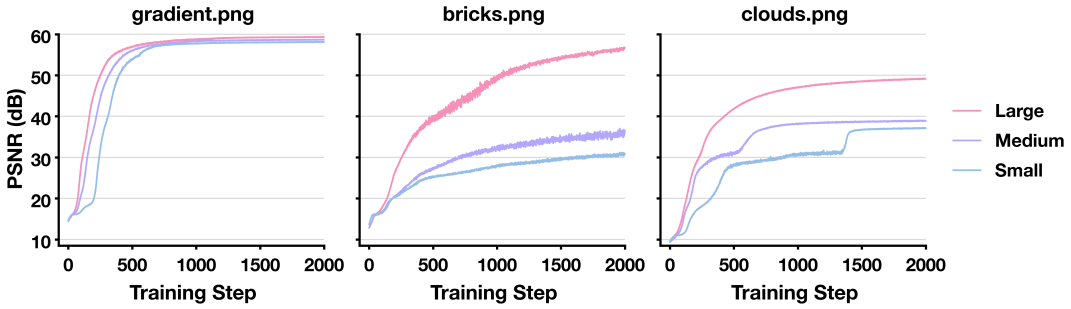
16 bits (2 bytes) does not evenly split into 3 channels. Since human perception is much more sensitive to the green channel, it is useful to provide this extra bit (effectively doubling the levels) to the green channel.

[P3] Feature Grid & Bilinear Lookup - in code
[P4] Decoder MLP - in code
[P5] Training Loop - in code
Modifications made:

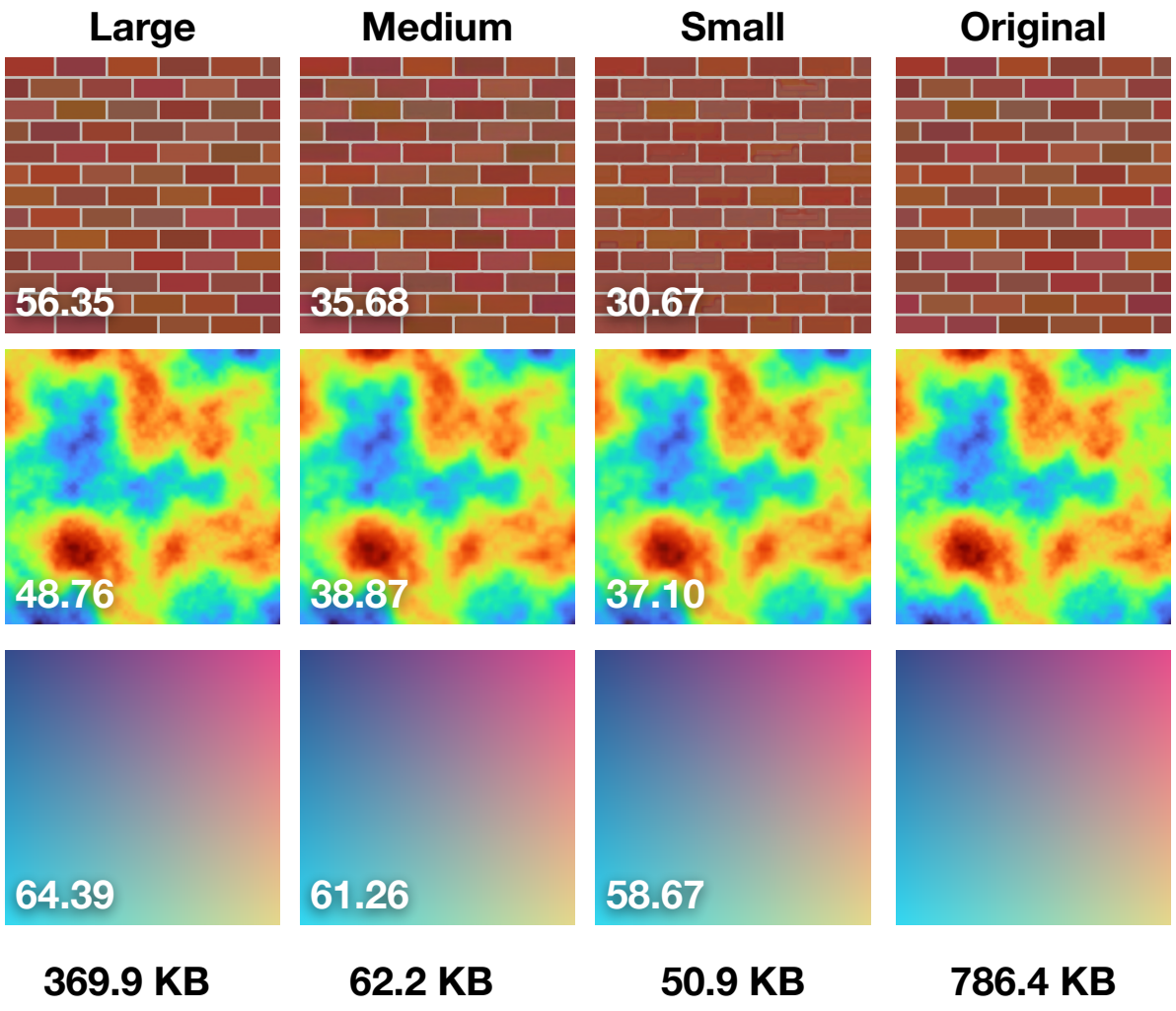
- Changed default learning rate to 1e-3. Explored using 3e-4 (which is my typical learning rate with Adam) but I found it to underfit with 2000 steps and 1e-2 was less stable in my testing.
- Fixed random seed of 42 throughout all runs.

[P6] Results & Comparison

Plot of PSNR over training for all 9 texture-and-architecture combinations

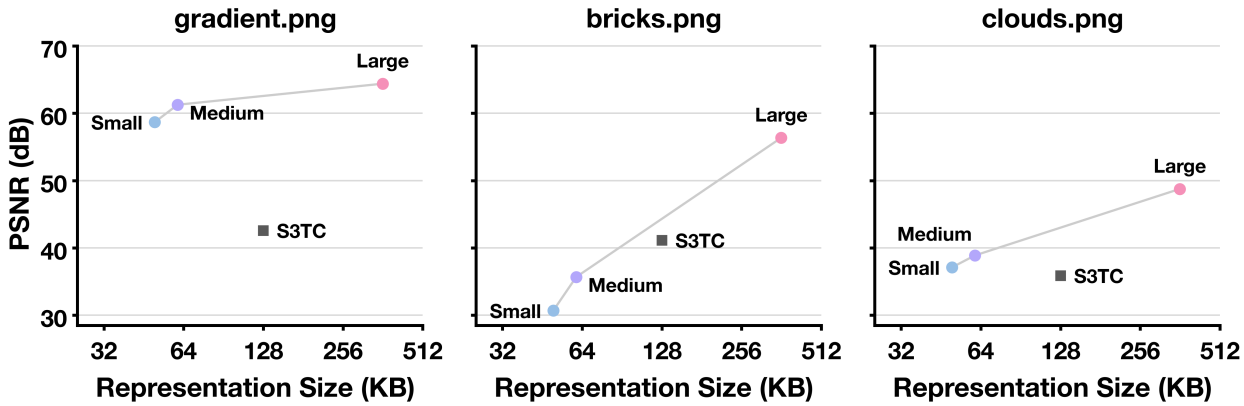


Compressed Textures with various MLP parameters. PSNR (dB) inset in white, Model/ Texture Size shown at the bottom in kilo bytes.



Model →	S3TC		Large		Medium		Small	
Image ↓	PSNR	CR	PSNR	CR	PSNR	CR	PSNR	CR
bricks.png	41.49	0.167	**56.35**	0.470	35.68	0.079	30.67	**0.065**
clouds.png	35.92	0.167	**48.76**	0.470	38.87	0.079	37.10	**0.065**
gradient.png	42.91	0.167	**64.39**	0.470	61.26	0.079	58.67	**0.065**

Comparison of compression performance (PSNR (in dB) / compression ratio (CR)) across various methods, models and images. Top performers are bolded and shown in green, second best methods are underlined and shown in yellow. Also plotted below.



For each texture, state which architecture performed best and justify why.

- For *bricks.png*, the Large MLP model performs the best, with a PSNR of 56.35 dB, followed by the S3TC baseline. The large mlp has both a larger decoder mlp as well as a larger set of feature grid resolutions, which helps it retain sharp detail in the brick texture, especially near the corners and edges. In contrast the smaller neural texture models lose fidelity near the corners (Medium model) and the color of the bricks (see artifacts in the small model output above). The S3TC baseline performs well due to the limited color palette of this texture, as well as the reduced visual frequencies of the texture.
- For *clouds.png,* the Large MLP performs the best, with a PSNR of 48.76 dB, followed by the Medium MLP. Notably, the difference in PSNR between the Medium model and the Small model is small, which indicates that the multi-resolution feature grid is not as critical for this texture. The bigger gap between the large and medium models comes from doubling the feature dimensionality from 2 to 4. To my eye, however, all the neural texture models do similarly well, and seem much better than the S3TC baseline.
- For *gradient.png,* once again the Large MLP performs the best with a PSNR of 64.39, closely followed by the medium model. Note how the smallest MLP tested is also performs quite well, with a PSNR of 58.67. This texture does not have high

frequency details, and for this reason is able to be modeled effectively by even the smallest mlp tested (along with the reduced feature grid and feature dimension).

For each texture, state whether its compression ratio beat S3TC.

- For all textures tested, only the Medium and small neural texture models beat the S3TC compression ratio. Compression ratio is only dependent on the model architecture, and not the input texture.

[P7] Quantization

Results are reported from only quantizing the grids.

Report PSNR before and after quantization.

Model →	Large		Medium		Small	
Image ↓	**Before Quant**	**After Quant**	**Before Quant**	**After Quant**	**Before Quant**	**After Quant**
bricks.png	56.35	56.13	35.68	35.50	30.67	30.60
clouds.png	48.76	48.70	38.87	38.86	37.10	37.06
gradient.png	64.39	61.48	61.26	58.41	58.67	56.36

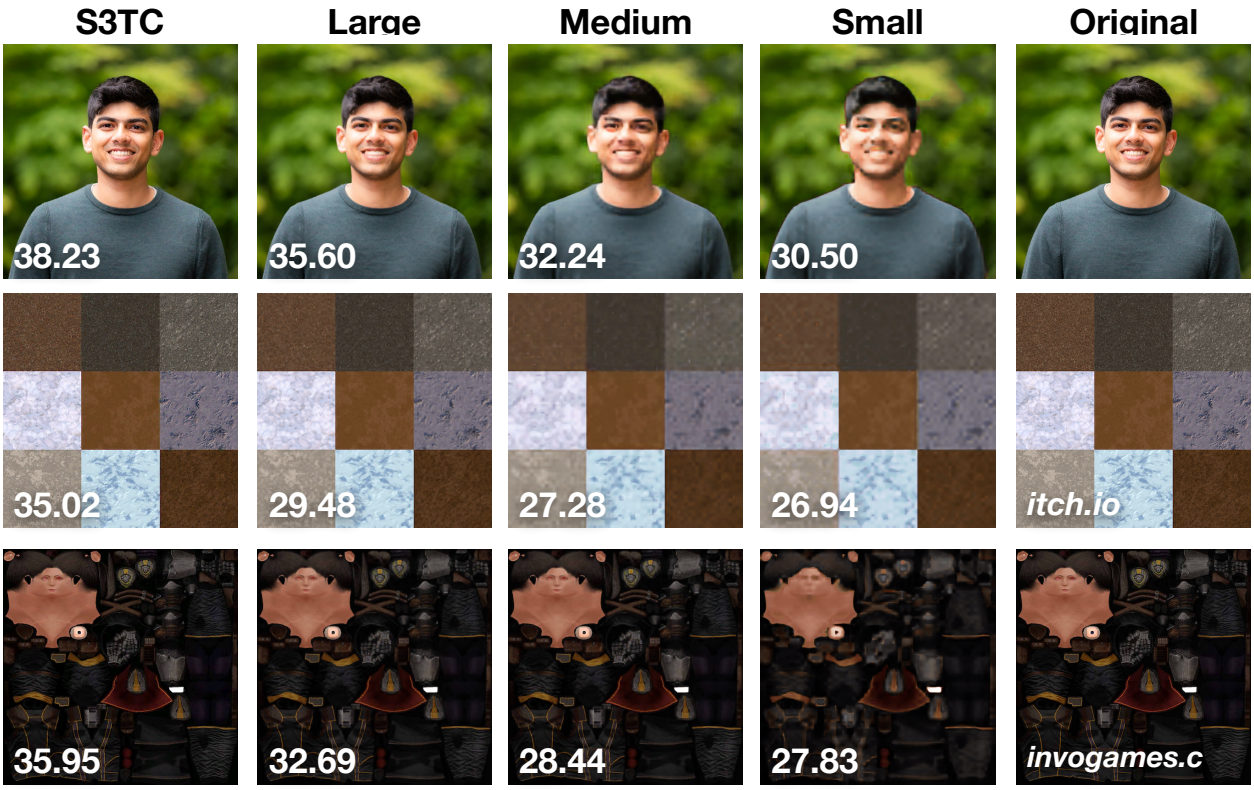
Report the new size and compression ratio.

Model	Size (KB)	Compression Ratio
Large	108.84	0.138
Medium	29.98	0.038
Small	26.38	0.034

Explain how much quality you lost and why.

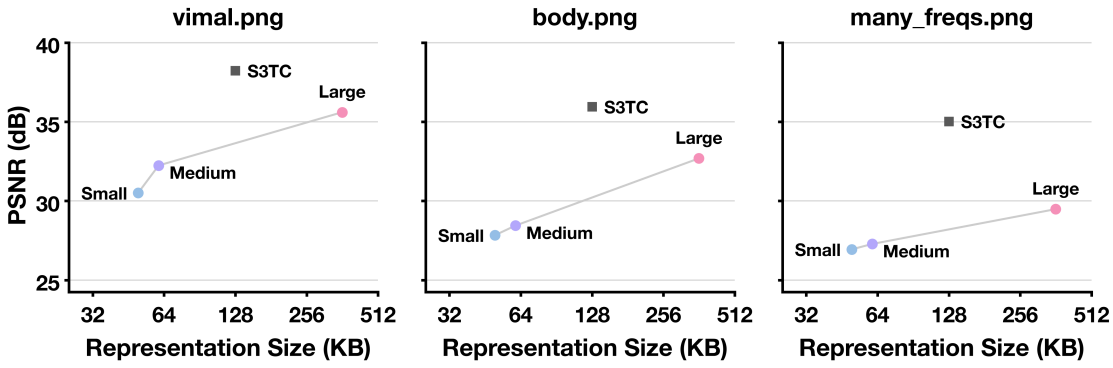
Loss in quality is negligible (< 1dB on average) for the *bricks.png* and the *clouds.png*. In the decoded textures, this loss in quality does not surface, which was surprising to me. On the other hand, the *gradient.png* loses more PSNR points. However, it's PSNR still remains higher than the other textures, and the quantized and compressed textures still look great to my eye. Example images can be found in the GitHub repo.

[P8] Your Own Textures



I picked the three textures shown above: my face, a texture sample pack, and a character texture. Each of these textures had different properties including various frequencies, realism, and fine details.

The PSNR for all the methods is reported. Surprisingly, contrary to the results with the previous textures, the S3TC methods seems to do the best both in terms of PSNR as well as my perceived quality. The next best method is consistently the Large neural texture model, however the compressed textures look “smoother” than the S3TC textures, indicating that the neural texture compression is not able to resolve high frequency features of the textures.



This can also be seen in the graphs above, where I’ve plotted the PSNR vs size. Note however, while the S3TC compression has the highest PSNR, it is not the smallest representation.